
Exame de Estruturas de Dados – 2010.Junho.26

Duração: 2:00 horas

Valorização: 12 valores.

1. Responda às seguintes questões:

a) Faça o estudo da complexidade do seguinte método:

```
void fun2(int n, int v[]) {
    int m, tmp;

    for (int i= 0; i< n-1; i++) {
        m= i;
        for (int j= i+1; j< n; j++) {
            if (v[m]<v[j])
                m= v[j];
        }
        tmp= v[i];
        v[i]= v[m];
        v[m]= tmp;
    }
}
```

b) Analise o código seguinte e diga quantas vezes escreve a mensagem “ora ora.”. Existe algo de errado com este código? Se sim, diga porquê.

```
int i;
for (i=1; i<=10; i++) {
    for (i=1; i<=5; i++)
        System.out.println("ora ora.");
}
```

c) Considere a sequência 12, 6, 9, 17, 20, 14, 6. Desenhe a árvore binária de pesquisa (com a relação de ordem $\leq$) criada com a sequência dada e a árvore binária de pesquisa que resulta de remover o nó raiz da árvore criada.

2. Listas, Pilhas e Filas

Considere um armazém de produtos onde são entregues fornecimentos de produtos e colocadas encomendas de clientes para esses produtos. Aqui vamos apenas considerar apenas fornecimentos e entregas de um dos produtos (por isso o nome do produto não será relevante). Os fornecimentos recebidos do produto são armazenados por empilhamento, ficando

no topo o fornecimento mais recente. Um fornecimento é caracterizado pela quantidade, custo unitário e nome do fornecedor.

Ao armazém chegam também encomendas de clientes para o produto considerado. Uma encomenda é caracterizada por uma quantidade e um nome de cliente. A ordem das encomendas deverá ser preservada, sendo sempre satisfeitas as encomendadas mais antigas.

O que aqui lhe pedimos é para concretizar a simulação da satisfação das encomendas em carteira. Claramente, os fornecimentos são bem representados por uma pilha e as encomendas por uma fila. Recomenda-se ainda o uso da classe seguinte:

```
class Armazem {
    Stack<Fornecimento> stock;
    Queue<Encomenda> pedidos;

    // métodos
}
```

- a) Implemente as classes **Fornecimento**, **Encomenda**, as classes genéricas **Stack<E>** e **Queue<E>**, indicando apenas as assinaturas dos métodos que as compõem.
- b) Escreva um método **void simulaEntregas()** da classe **Armazem** que procurará simular a execução das encomendas colocadas pelos clientes fazendo uso dos produtos em armazém. Para satisfazer uma encomenda poderá usar mais do que um fornecimento. Na satisfação das encomendas, há um lucro associado que corresponde a 10% de ganho.

No final da simulação deverá indicar o total de clientes atendidos e o lucro total conseguido. Deverá ainda escrever a mensagem “Todas encomendas satisfeitas” caso a fila de pedidos fique vazia ou “Produto esgotado” caso não tenha conseguido satisfazer todos os pedidos.

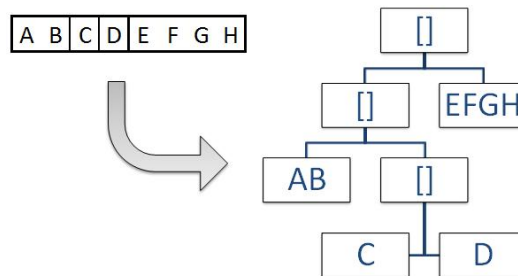
3. Árvores binárias

A NASA possui um sistema para processamento das ondas captadas pelos seus radiotelescópios. No programa actual essas ondas encontram-se armazenadas sob a forma de strings. Como forma de estudar as ondas procura-se dedicar atenção a partes específicas na medida dos recursos que existem. Para tal pensou-se em rever o program de modo a passar a usar uma árvore binária que divide e armazena a string, considerando que:

- as substrings devem ser colocadas somente nas folhas e não nos nós intermédios da árvore;
- o método **boolean dividir(String s)** diz-nos se num dado momento a string em análise deve ser subdividida ou não;
- o método **String divide(String s, int lado)** retorna o lado esquerdo da string quando **lado=0**, e retorna o lado direito quando **lado=1**.

O algoritmo a seguir será o seguinte. Dada uma sequência inicial (ou string), se for para dividir, divide-a ao meio a string em duas partes esquerda e direita. De seguida deverá continuar o processo de divisão na parte esquerda e na parte direita. Não se subdivide sequências de tamanho 1.

A figura seguinte ilustra uma situação em que a sequência inicial **ABCDEFGH** foi dividida em **ABCD** + **EFGH** na primeira divisão; depois a primeira subsequência foi de novo dividida em **AB** + **CD**, finalmente a última string foi mais uma vez sub-dividida em **C** + **D**. Note que nos nós intermédios não são armazenadas sequências.



- Escreva as `BTNode` e `BTree` para representar, respectivamente, um nó de uma árvore e uma árvore propriamente dita. Inclua os métodos construtores de ambas as classes.
- Escreva o código que implementa na classe `BTree` um método `void printTree()` que deverá imprimir as folhas da árvore, uma por linha e da esquerda para a direita, e para cada folha o caminho da raiz até esse nó, usando `E` quando vai para o filho esquerdo e `D` quando vai para o filho direito.

A título de exemplo, para a árvore da figura o resultado seria:

```
AB caminho: EE
C caminho: EDE
D caminho: EDD
EFGH caminho: D
```

- Escreva o método `void createTree(String s)` que criar a árvore que representa a string `s`.

Faça uso dos seguintes métodos que supõe implementados:

- `boolean adividir(String s)` – retorna `true` se a string dever ser dividida e `false` caso contrário.
- `String divide(String s, int lado)` – retorna a metade esquerda da string `s` se `lado==0`; retorna a metade do lado direito se `lado==1`.

Como sugestão, use uma metodologia de pesquisa em profundidade primeiro. Será o método `adividir()` que pode impedir a continuação do processo de divisão.